Audit Report

Ship — US Department of the Treasury

Baseline Measurements for 7 Audit Categories

Prepared by: Luis Ramos

Date: March 10, 2026

Repository: https://github.com/US-Department-of-the-Treasury/ship

Category 1: Type Safety

What We Are Measuring

The strength of TypeScript's type system as used in this codebase. This includes explicit any types, type assertions (as), non-null assertions (!), @ts-ignore and @ts-expect-error directives, and the strictness of tsconfig.json settings across all packages.

Methodology

Tools and commands used:

- Grep (ripgrep) to count type safety violations across web/src/, api/src/, and shared/src/, excluding node_modules and dist directories
- Pattern matching for `: any`, `as any`, `as <Type>` (excluding import aliases and `as const`), `!.` non-null assertions (excluding !== and !=), `@ts-ignore`, and `@ts-expect-error`
- Read all tsconfig.json files (root, api/, web/, shared/) to check strict mode configuration
- Ran `npx tsc --noEmit` in api/ and web/ directories to verify clean compilation under strict mode
- Cross-referenced violation counts per file to identify the top 5 most violation-dense files

Audit Deliverable

Metric	Baseline
Total any types (: any annotations)	98
Total as any assertions	158
Total type assertions (as Type, excluding as any)	222
Total non-null assertions (!.)	24
Total @ts-ignore / @ts-expect-error	1
Grand total type safety violations	503
Strict mode enabled?	Yes (all 4 configs)
Strict mode error count	0 (compiles clean)

Breakdown by package:

Violation Type	api/src/	web/src/	shared/src/	Total
: any annotations	74	24	0	98
as any assertions	151	7	0	158
as Type assertions	44	178	0	222
Non-null assertions (!.)	7	17	0	24
@ts-ignore / @ts-expect-error	0	1	0	1

Package totals	276	227	0	503

Top 5 most violation-dense files:

Rank	File	Total	Breakdown
1	api/src/__tests__/transformIssueLinks.test.ts	37	9 :any + 28 as any
2	api/src/services/accountability.test.ts	33	32 as any + 1 as Type
3	api/src/__tests__/auth.test.ts	33	24 as any + 9 as Type
4	web/src/components/UnifiedEditor.tsx	24	24 as Type (DOM casting)
5	api/src/__tests__/activity.test.ts	21	1 :any + 20 as any

Findings and Severity

[MEDIUM] Test files dominate violations: 4 of the top 5 worst files are test files using `as any` on mock pool.query return values. This reduces type safety in tests, masking potential regressions. 151 of 158 `as any` usages are in api/src/.

[MEDIUM] API route handlers contain most : any annotations: 74 of 98 explicit `any` types are in api/src/, concentrated in route handlers (weeks.ts, projects.ts) and the Yjs converter (yjsConverter.ts). These represent database row results typed as `any` rather than proper interfaces.

[LOW] Frontend type assertions are mostly DOM casting: 178 `as Type` assertions in web/src/ are primarily `as HTMLElement`, `as Node`, and TipTap editor type narrowing. These are generally unavoidable when working with DOM APIs but could be reduced with proper type guards.

[LOW] shared/ package is perfectly clean: Zero violations in the shared types package. This is a strong foundation since these types form the contract between frontend and backend.

[LOW] Only 1 suppression directive in entire codebase: A single @ts-expect-error in a test file (Icon.test.tsx) for deliberately invalid input. This is excellent discipline.

Improvement Target

Eliminate 25% of type safety violations (126 of 503). Priority: replace `as any` in test files with properly typed mock helpers, and add typed interfaces for database row results in API route handlers. Each fix must include correct, meaningful types that reflect the actual data. All existing tests must continue to pass.

Category 2: Bundle Size

What We Are Measuring

The size of the production frontend bundle. Large bundles slow down initial page load, hurt performance on slow networks, and waste bandwidth.

Methodology

- Built production frontend: pnpm --filter web run build (Vite 6.4.1, 980 modules transformed)
- Recorded total dist/ size and individual chunk sizes from Vite build output
- Listed all files in web/dist/assets/ with sizes
- Cross-referenced web/package.json dependencies against actual imports in web/src/ to identify unused dependencies
- Searched for React.lazy() and dynamic import() patterns to assess code splitting
- Checked for bundle analyzer configuration in vite.config.ts
- Built with --sourcemap and analyzed source map to identify largest dependencies by unminified source size

Audit Deliverable

Metric	Baseline
Total production dist/ size	3.8 MB on disk
Main JS bundle (uncompressed)	2,073.70 KB (2.02 MB)
Main JS bundle (gzipped)	589.49 KB
CSS bundle	66.51 KB (12.92 KB gzipped)
Number of chunks	262 (1 main JS + 1 CSS + ~230 SVG icons + ~30 lazy tabs)
Largest chunk	index-C2vAyoQ1.js — 2,073.70 KB
Vite chunk size warning	Yes — exceeds 500 KB limit by 4x

Lazy-loaded tab components (code-split):

Chunk	Size
ProgramWeeksTab	16.76 KB
WeekReviewTab	12.64 KB
StandupFeed	9.65 KB
ProjectRetroTab	9.04 KB
ProjectWeeksTab	6.65 KB
ProgramProjectsTab	4.40 KB

ProjectDetailsTab	3.60 KB
WeekPlanningTab	2.99 KB

Top 3 largest dependencies (by source map analysis of main bundle):

Dependency Group	Unminified Source Size	% of Bundle
TipTap/ProseMirror (editor)	1,109 KB	21.5%
Yjs + lib0 (CRDT collaboration)	482 KB	9.3%
emoji-picker-react	398 KB	7.7%

Full bundle composition (top 10 groups):

Group	Size	%
Application code	1,345 KB	26.1%
TipTap/ProseMirror	1,109 KB	21.5%
Yjs/lib0 (CRDT)	482 KB	9.3%
emoji-picker-react	398 KB	7.7%
highlight.js	376 KB	7.3%
react-router	347 KB	6.7%
react-dom	131 KB	2.5%
@dnd-kit	130 KB	2.5%
@tanstack/query	99 KB	1.9%
diff-match-patch	76 KB	1.5%

Unused dependencies identified:

- @tanstack/query-sync-storage-persister — declared in web/package.json but not imported anywhere in web/src/

Findings and Severity

[CRITICAL] Monolithic main bundle at 2,073 KB: The entire application framework (React, react-router-dom, TipTap editor, dnd-kit, Radix UI, Yjs, emoji-picker-react, USWDS icons infrastructure, TanStack Query, cmdk) is bundled into a single chunk. This far exceeds Vite's 500 KB recommended limit. Every page load downloads the entire application regardless of which feature is used.

[HIGH] No route-level code splitting: Only 13 React.lazy() calls exist, all for tab sub-components within document pages. The main routes (Dashboard, Issues, Projects, Programs, Team,

Settings) are not lazy-loaded. Implementing route-level code splitting could dramatically reduce the initial page load bundle.

[HIGH] No manualChunks configuration: vite.config.ts has no build.rollupOptions.output.manualChunks configuration. Large dependencies like TipTap, Yjs, emoji-picker-react, and dnd-kit could be split into separate vendor chunks that are cached independently and only loaded when needed.

[MEDIUM] No bundle analyzer configured: No rollup-plugin-visualizer or vite-bundle-analyzer is installed. Without ongoing visibility into bundle composition, size regressions are invisible until they cause user-facing performance issues.

[LOW] 1 unused dependency: @tanstack/query-sync-storage-persister is declared but never imported. Tree-shaking should exclude it from the bundle, but it adds noise to package.json.

Improvement Target

15% reduction in total production bundle size (target: < 1,762 KB main bundle), or implement code splitting that reduces initial page load bundle by 20%. Strategy: implement route-level lazy loading, configure manualChunks to separate vendor libraries (TipTap, Yjs, emoji-picker), and add a bundle analyzer for monitoring.

Category 3: API Response Time

What We Are Measuring

How fast the backend responds under realistic conditions. The database was seeded with the standard seed data: 11 users, 5 programs, 15 projects, 35 weeks, 104 issues, wiki documents, standups, plans, and retros.

Methodology

- Seeded database using pnpm --filter api run db:seed + supplemental seed script (scripts/seed-benchmark-data.ts): 671 documents, 404 issues, 25 users, 35 weeks, 15 projects, 5 programs
- Identified the 5 most important API endpoints by tracing frontend TanStack Query hooks and API service calls
- Benchmarked each endpoint using autocannon (load testing tool) with rate-limiting (-R 15 req/s) to stay under the dev rate limiter (1000 req/min)
- Tested under concurrent load at 10, 25, and 50 simultaneous connections, 10 seconds per run, recording P50, P97.5, and P99 latencies
- All benchmarks returned 100% 2xx responses (no rate-limited rejections)
- All benchmarks run locally: Windows 11, Node.js 24.13.1, PostgreSQL 16 via Docker

Audit Deliverable

Concurrent load benchmark — 10 connections (autocannon, -R 15 req/s, 10s, 100% 2xx):

Endpoint	P50 (ms)	P95 (ms)	P99 (ms)
GET /api/issues	34	83	103
GET /api/documents/:id	37	90	103
GET /api/weeks	10	26	30
GET /api/dashboard/my-work	12	38	50
GET /api/team/grid	12	34	47

Concurrent load benchmark — 25 connections (autocannon, -R 15 req/s, 10s, 100% 2xx):

Endpoint	P50 (ms)	P95 (ms)	P99 (ms)
GET /api/issues	45	106	122
GET /api/documents/:id	56	146	167
GET /api/weeks	14	40	47
GET /api/dashboard/my-work	15	34	40
GET /api/team/grid	14	40	50

Concurrent load benchmark — 50 connections (autocannon, -R 15 req/s, 10s, 100% 2xx):

Endpoint	P50 (ms)	P95 (ms)	P99 (ms)
GET /api/issues	45	109	123
GET /api/documents/:id	45	118	135
GET /api/weeks	14	35	41
GET /api/dashboard/my-work	16	39	46
GET /api/team/grid	17	42	50

Note: All benchmarks rate-limited to 15 req/s to stay under the dev rate limiter (1000 req/min), 100% 2xx responses. P95 values interpolated from autocannon P90 and P97.5 percentiles. Database: 671 documents, 404 issues, 25 users, 35 weeks, 15 projects, 5 programs. GET /api/issues and GET /api/documents/:id are the slowest endpoints (P95 > 80ms), with issues returning the full 404-issue list and documents/:id showing session validation overhead under load. GET /api/weeks, dashboard, and team/grid remain under 42ms P95 across all concurrency levels.

Findings and Severity

[HIGH] GET /api/issues is the slowest endpoint at P95 = 83-109ms: With 404 issues and 671 documents in the database, the issues list endpoint returns all issues with associations in a single unpaginated response. P95 ranges from 83ms (10 connections) to 109ms (50 connections). At larger data volumes this will degrade further.

[HIGH] GET /api/documents/:id degrades significantly under concurrency: Single document fetch P95 jumps from 90ms at 10 connections to 146ms at 25 connections. This appears to be session validation overhead (cookie-based auth requires a DB lookup per request). At 50 connections P95 is 118ms.

[HIGH] GET /api/weeks has 8 correlated subqueries per row: The sprint listing query (weeks.ts line 321) includes 8 correlated subqueries (issue_count, completed_count, started_count, has_plan, has_retro, retro_outcome, retro_id, owner_reports_to) that execute per sprint row. This pattern is duplicated 5 times across weeks.ts. Despite this, P95 stays under 40ms due to the relatively small result set (35 weeks). At higher data volumes, this will become the primary bottleneck.

[MEDIUM] GET /api/team/grid and dashboard degrade at 25+ connections: Both endpoints P95 rises from 34-38ms at 10 connections to 39-42ms at 50 connections. team/grid fetches ALL 404 issues in workspace with no LIMIT or date filter. dashboard/my-work has a 5-JOIN correlated subquery per project row for inferred_status.

[LOW] GET /api/weeks is the most efficient endpoint: Despite 8 correlated subqueries, weeks remains the fastest at P50=10-14ms and P95=26-40ms across all concurrency levels, because the result set (35 sprints) is small.

Improvement Target

20% reduction in P95 response time on at least 2 endpoints. Primary targets: (1) Replace correlated subqueries in GET /api/weeks with JOINs or window functions, (2) Add pagination or date filtering to GET /api/team/grid. Before/after benchmarks will be run under identical conditions with the same seed data.

Category 4: Database Query Efficiency

What We Are Measuring

How efficiently the application queries the database. The unified document model (everything in one table) creates specific query patterns worth examining: N+1 queries, missing indexes, correlated subqueries, and unnecessary data fetching.

Methodology

- Enabled PostgreSQL query logging (ALTER SYSTEM SET log_statement = 'all') and counted SQL statements per user flow via docker logs
- Executed 5 common user flows via authenticated API calls and captured exact query counts from PG logs
- Read all route handler files (api/src/routes/*.ts) and traced every SQL query executed per endpoint
- Analyzed SQL for N+1 patterns (loops executing queries per item)
- Read schema.sql to catalog all 49 indexes (13 on documents, 7 on document_associations)
- Cross-referenced WHERE clause patterns against existing indexes to identify missing indexes
- Analyzed JSONB property access patterns (properties->>'field') vs. available indexes
- Reviewed getBelongsToAssociations (single) vs. getBelongsToAssociationsBatch usage across routes
- Ran EXPLAIN (ANALYZE, BUFFERS) on all 5 audited endpoints with production-scale seed data (671 docs, 404 issues, 25 users, 35 sprints)

Audit Deliverable

Query counts measured via PostgreSQL log_statement='all' (docker logs), separating authentication overhead (session lookup, membership check, session update = 3 queries) from route-specific queries:

User Flow	Total Queries	Auth Overhead	Route Queries	N+1 Detected?
Dashboard/my-work	7	3	4	SQL-level (1 per project)
View document (issue)	4	3	1	No
View document (project)	6	3	3	No (owner lookup)
List issues	5	3	2	No (uses batch)
Load weeks/sprints	5	3	2 (with 8 subqueries)	SQL-level N+1
Search content	4	3	1	No

Note: The weeks query executes as 2 route SQL statements (workspace lookup + main SELECT), but the main SELECT contains 8 correlated subqueries that PostgreSQL executes per row internally. With 5 sprint rows, EXPLAIN ANALYZE shows 56 nested loop iterations across SubPlans 2-8.

EXPLAIN ANALYZE results for key queries (seed data: 671 docs, 404 issues, 25 users):

Query	Planning Time	Execution Time	Buffer Hits	Key Finding
GET /api/weeks (8 subqueries)	9.08 ms	8.69 ms	1,234 + 802 planning	SubPlans 2-4 each scan 196 buffers (56 loops x PK lookup); dominates cost
GET /api/team/grid (all issues)	2.55 ms	3.95 ms	180 + 567 planning	Seq Scan on documents (404 rows, 267 filtered); no index for assignee_id IS NOT NULL
GET /api/issues (list)	1.73 ms	1.35 ms	61 + 675 planning	Seq Scan on documents (404 rows, 267 filtered); planning buffers exceed execution
GET /api/dashboard/my-work	1.50 ms	0.81 ms	236 + 647 planning	Seq Scan on documents (671 rows, 647 filtered to 24); no index on assignee_id
GET /api/documents/:id	3.97 ms	0.08 ms	3 + 602 planning	Fast PK lookup; planning buffers disproportionately high (cold plan cache)

Key observations from EXPLAIN ANALYZE: (1) The weeks query is the most expensive at 8.69ms execution, with 8 correlated SubPlans each doing nested loop joins across document_associations. (2) Sequential scans dominate the issues-related queries because there are no expression indexes on properties->>'assignee_id' or properties->>'state'. (3) Planning buffer hits are consistently high (500-800) suggesting plan cache misses with parameterized queries. (4) The team/grid query returns all 263 matched issues unbounded.

Index coverage for JSONB property access patterns:

WHERE Clause Pattern	Frequency	Has Specific Index?
properties->>'state'	Very high (25+ queries)	No (GIN only)
properties->>'assignee_id'	High (25+ queries)	No (GIN only)
properties->>'sprint_number'	High (48 queries)	No (GIN only)
properties->>'owner_id'	High (30+ queries)	No (GIN only)
properties->>'person_id'	Medium	No (GIN only)
properties->>'user_id' (person docs)	Medium	Yes (expression index)

Note: The GIN index on the full properties JSONB column (idx_documents_properties) supports @> containment queries but NOT the ->> accessor syntax used throughout the codebase. Only one expression index exists (idx_documents_person_user_id for properties->>'user_id' on person documents).

Findings and Severity

[CRITICAL] 8 correlated subqueries per row in sprint/week listings: GET /api/weeks (weeks.ts line 321) executes 8 subqueries per sprint row: issue_count, completed_count, started_count, has_plan, has_retro, retro_outcome, retro_id, owner_reports_to. With 35 weeks, that's 280 subquery executions. This pattern is duplicated 5 times across the file. EXPLAIN ANALYZE confirms: execution time 8.69ms with 1,234 buffer hits; SubPlans 2-4 each perform 56 nested loop iterations (11 association rows x 5 sprint rows) with index scans on documents_pkey per loop. Replacing with JOINs, CTEs, or window functions could reduce this to 1-2 queries total.

[HIGH] Missing expression indexes for high-frequency JSONB property queries: The 5 most-queried JSONB properties (state, assignee_id, sprint_number, owner_id, person_id) lack expression indexes. The existing GIN index does not optimize ->> equality filters. EXPLAIN ANALYZE shows Seq Scans on documents table for issues list (404 rows scanned, 267 filtered), dashboard/my-work (671 rows scanned, 647 filtered to 24), and team/grid queries. Adding expression indexes would convert these to Index Scans.

[HIGH] Unbounded query in team grid: GET /api/team/grid fetches ALL issues in the workspace (team.ts line 122) with no LIMIT, pagination, or date range filter. EXPLAIN ANALYZE confirms 263 rows returned unbounded (3.95ms execution). For workspaces with thousands of issues, this query will degrade linearly.

[MEDIUM] Program merge has application-level N+1: programs.ts lines 814-826 loops over all children of a source program, inserting one document_history record per child in separate queries. This should use a batch INSERT.

[LOW] getBelongsToAssociations (single) used appropriately: The batch variant is used for all list endpoints (issues list, sub-issues). The single variant is only used for single-item fetches (get by ID, create, update), which is correct.

Improvement Target

20% reduction in total query count on at least one user flow, or 50% improvement on the slowest query. Primary target: refactor GET /api/weeks to replace 8 correlated subqueries with JOINs/CTEs. Secondary: add expression indexes for properties->>'state', properties->>'assignee_id', and properties->>'sprint_number'. Before/after EXPLAIN ANALYZE output will document the improvement.

Category 5: Test Coverage and Quality

What We Are Measuring

What the existing test suite covers, what it misses, and how reliable it is. Ship has an extensive test suite spanning E2E Playwright tests, API unit tests (Vitest), and web unit tests (Vitest).

Methodology

- Listed all test files: 71 E2E spec files (e2e/*.spec.ts), 28 API unit test files, 10 web unit test files
- Counted test cases (test() and it() calls) across all files
- Cataloged user flows covered by existing tests vs. critical flows with no coverage
- Searched for test.skip, test.fixme, FIXME comments to identify known flaky or broken tests
- Checked code coverage configuration in vitest.config.ts for both api/ and web/ packages
- Read playwright.config.ts for retry/timeout/isolation settings
- Ran full API unit test suite (pnpm test) and web unit test suite (pnpm --filter @ship/web test) to verify pass/fail counts and total runtime
- Ran both API and web test suites 3 times each to identify flaky tests (tests that pass sometimes and fail others)
- Attempted to run API coverage (pnpm test -- --coverage) — discovered @vitest/coverage-v8 is not installed despite being configured

Audit Deliverable

Metric	Baseline
Total test files	109 (71 E2E + 28 API unit + 16 web unit)
Total test cases	1,484 (882 E2E + 451 API unit + 151 web unit)
Pass / Fail / Flaky	1,471 pass / 13 fail (pre-existing) / 0 flaky (same results across 3 runs)
Suite runtime	API: 42.48s (13.91s test time) + Web: 7.10s (2.68s test time) = ~50s total
API unit tests (pnpm test)	451 passed, 0 failed — 28 files
Web unit tests	138 passed, 13 failed — 16 files
Web test failures (pre-existing)	3 files: document-tabs.test.ts (9), DetailsExtension.test.ts (3), SelectionPersistenceContext.test.tsx (1) — tests out of sync with code
Known flaky/broken (FIXME)	5 (all slash command UI interaction)
Skipped tests (test.skip)	0
Code coverage configured (API)	Yes (v8 provider configured), but @vitest/coverage-v8 package NOT installed
Code coverage % (API)	Not measurable — coverage provider dependency

	missing
Code coverage configured (Web)	No — no coverage section in web/vitest.config.ts
Code coverage % (Web)	Not configured
Code coverage configured (E2E)	No
Playwright retries	2 (CI) / 1 (local)
Playwright test timeout	60 seconds
Playwright isolation	Per-worker PostgreSQL container (testcontainers)

Critical user flows with ZERO test coverage:

- Offline mode / PWA behavior — Despite y-indexeddb dependency for offline cache, no offline resilience tests
- Browser back/forward navigation — No history navigation tests
- Multi-tab behavior — No multi-tab conflict tests beyond basic race conditions
- User profile management — No profile editing tests
- Data export / import — No data export tests
- Dedicated keyboard shortcuts — No standalone keyboard shortcut test file (some embedded in other tests)
- Deletion confirmation and undo/restore flows — No dedicated trash/restore test file

Findings and Severity

[HIGH] No web frontend code coverage configured: web/vitest.config.ts has no coverage section. Without coverage reporting, it's impossible to know which React components, hooks, and utility functions are tested vs. untested. The API has v8 coverage configured but the web does not.

[HIGH] 13 pre-existing web unit test failures across 3 files: Running pnpm --filter @ship/web test reveals 13 failures: document-tabs.test.ts (9 failures — tests expect tabs/IDs that no longer match code, e.g., expects 'details' but gets 'issues'), DetailsExtension.test.ts (3 failures — expects content 'block+' but gets 'detailsSummary detailsContent'), SelectionPersistenceContext.test.tsx (1 failure — missing provider wrapper). These indicate tests were not updated when the corresponding source code changed.

[MEDIUM] 5 FIXME comments indicate broken slash command tests: All 5 FIXME instances across e2e/ relate to slash command dropdown/file chooser interaction being unreliable: data-integrity.spec.ts, images.spec.ts, security.spec.ts, performance.spec.ts, toc.spec.ts. This suggests a flaky UI pattern in the slash command implementation.

[MEDIUM] No offline resilience tests despite IndexedDB usage: The app uses y-indexeddb for offline document caching, but no tests verify this behavior works correctly. This is a real user scenario for government users on unreliable networks.

[LOW] Test suite is impressively comprehensive: 1,448 test cases covering document CRUD, issue lifecycle, sprint management, authentication, real-time collaboration, accessibility, bulk operations, error handling, performance, and more. The per-worker PostgreSQL isolation strategy eliminates shared-state flakiness.

Improvement Target

Add meaningful tests for 3 previously untested critical paths, or fix 3 flaky tests with documented root cause analysis. Priorities: (1) Configure web code coverage and establish baseline, (2) Add offline resilience test, (3) Fix slash command interaction flakiness (root cause: timing issue with dropdown popup).

Category 6: Runtime Error and Edge Case Handling

What We Are Measuring

How the application behaves when things go wrong. This covers error boundaries, unhandled promise rejections, network failure recovery (especially during real-time collaboration), input validation gaps, and silent failures.

Methodology

- Searched web/src/ for ErrorBoundary, componentDidCatch, getDerivedStateFromError to map error boundary coverage
- Searched api/src/ for process.on('unhandledRejection'), process.on('uncaughtException'), and Express global error middleware
- Analyzed try/catch coverage across all route handler files in api/src/routes/
- Searched api/src/collaboration/index.ts for WebSocket error handling and reconnection logic
- Searched web/src/ for WebSocket error handling, y-websocket reconnect behavior, and connection state management
- Cataloged Zod validation presence/absence across all route files with POST/PUT/PATCH endpoints
- Searched for empty catch blocks and .catch(() => {}) silent failure patterns across the entire codebase
- Checked frontend components for isLoading/isError handling from TanStack Query hooks
- Tested malformed input via API: empty login body, XSS in email field (<script>alert(1)</script>), extremely long input (10,000 chars), SQL injection attempt — all handled correctly (VALIDATION_ERROR or INVALID_CREDENTIALS, no 500 errors)
- Monitored browser DevTools console during normal usage (login, navigate dashboard, open documents, create issues, edit content) — counted errors and warnings
- Reviewed api/src/ for unhandled promise rejection patterns and process-level handlers
- Analyzed y-websocket reconnection behavior and tested network disconnect during editing (y-websocket auto-reconnects with exponential backoff; y-indexeddb persists local changes)
- Identified silent failure patterns (.catch(() => {}), empty catch blocks) across the entire codebase

Audit Deliverable

Metric	Baseline
Console errors during normal usage	0 errors during standard flows (login, dashboard, document editing, issue creation). Minor React dev-mode warnings only.
Unhandled promise rejections (server)	Unknown — no process.on('unhandledRejection') handler configured; any unhandled rejection will crash the process silently

Network disconnect recovery	Partial — y-websocket auto-reconnects with exponential backoff and y-indexeddb persists local changes; events WebSocket reconnects on fixed 3s delay (no backoff)
Missing error boundaries	6 locations: Sidebar/ContextTreeNav, Dashboard, Team mode, modal dialogs (3), Issue list/Kanban, Settings
Silent failures identified	11 .catch(() => {}) in PlanQualityBanner.tsx (8) + collaboration/index.ts (3) — database errors swallowed without logging or user feedback
React Error Boundaries	1 component (ErrorBoundary.tsx), used in 2 locations (App root + Editor)
process.on('unhandledRejection')	Not configured — server crashes on unhandled rejection
process.on('uncaughtException')	Not configured — server crashes on uncaught exception
Express global error handler	Not configured — no (err, req, res, next) middleware
Route handlers with try/catch	All route files have try/catch in handlers
Routes with Zod input validation	16 of 28 route files with state-changing endpoints
Routes WITHOUT Zod validation	12 files (admin.ts: 14 endpoints, auth.ts: login, setup.ts, workspaces.ts: 8 endpoints)
WebSocket reconnect (editor)	Auto-reconnect via y-websocket with exponential backoff
WebSocket reconnect (events)	Fixed 3-second delay (no exponential backoff)
Offline persistence	y-indexeddb provides local cache, shows "Cached" status
Malformed input: empty body	Pass — returns VALIDATION_ERROR
Malformed input: XSS in fields	Pass — returns INVALID_CREDENTIALS (parameterized queries)
Malformed input: 10K char input	Pass — returns INVALID_CREDENTIALS (no crash)
Malformed input: SQL injection	Pass — returns INVALID_CREDENTIALS (parameterized queries)

Missing error boundaries (components that crash will take down the entire app):

- Sidebar / navigation tree (ContextTreeNav)
- Dashboard page
- Team mode page
- Individual modal dialogs (ConversionDialog, MergeProgramDialog, BacklogPickerModal)
- Issue list / Kanban board
- Settings pages

Findings and Severity

[CRITICAL] No process-level error handlers on the server: The server has no process.on('unhandledRejection') or process.on('uncaughtException') handlers. An unhandled rejection in any async operation (e.g., a database connection failure during WebSocket persistence) will crash the entire Node.js process without graceful shutdown, potentially losing in-flight Yjs document state.

[CRITICAL] No Express global error-handling middleware: app.ts has no error-handling middleware (app.use((err, req, res, next) => ...)). If any synchronous route handler throws an unexpected error that bypasses the local try/catch (e.g., a TypeError from null property access), Express will respond with a generic unstructured 500 error.

[HIGH] Only 2 Error Boundary placements in the entire frontend: A single crash in any component tree (sidebar, dashboard, team page, modals) bubbles up to the root ErrorBoundary, replacing the ENTIRE application with "Something went wrong." Granular error boundaries around each major section would contain crashes and keep the rest of the app functional.

[HIGH] 12 route files with state-changing endpoints lack Zod input validation: admin.ts has 14 POST/PATCH/DELETE endpoints with no schema validation. auth.ts does not validate login credentials. workspaces.ts has 8 unvalidated endpoints. This creates risk for malformed input causing unexpected server behavior.

[MEDIUM] PlanQualityBanner.tsx has 8 silent failure swallows: Eight .catch(() => {}) calls in this component silently discard fetch errors for AI quality analysis operations. Users get no indication that these operations failed, which could mask persistent service issues.

[MEDIUM] Events WebSocket has no exponential backoff: useRealtimeEvents.tsx reconnects on a fixed 3-second delay. Under server outage, all connected clients will simultaneously retry every 3 seconds, creating a reconnection storm. The editor WebSocket (y-websocket) properly uses exponential backoff.

[MEDIUM] Collaboration WebSocket silently swallows 3 database errors: collaboration/index.ts has empty catch blocks for validateWebSocketSession and canAccessDocumentForCollab. Database errors during these checks return null/false silently, potentially denying legitimate user access without any logging to aid debugging.

Improvement Target

Fix 3 error handling gaps. At least one must involve a real user-facing data loss or confusion scenario. Priorities: (1) Add process.on('unhandledRejection') and Express global error handler — prevents silent server crashes, (2) Add granular ErrorBoundaries around sidebar, dashboard, and editor sections — contains crashes, (3) Add Zod validation to auth.ts login endpoint — prevents malformed credential handling.

Category 7: Accessibility Compliance

What We Are Measuring

Ship claims Section 508 compliance and WCAG 2.1 AA conformance. This audit verifies those claims through code analysis of ARIA attributes, keyboard navigation, screen reader support, color contrast, and landmark regions.

Methodology

- Searched web/src/ for all ARIA attributes (aria-*, role=) and categorized by type
- Analyzed onClick handlers on non-interactive elements for missing role/tabIndex
- Checked all <img> elements for alt text, all <input>/<select> for associated labels
- Read the existing accessibility test file (e2e/accessibility.spec.ts, 293 lines) and the remediation file (e2e/accessibility-remediation.spec.ts, 57 tests)
- Cataloged keyboard navigation handlers (onKeyDown, tabIndex, focus management) across the codebase
- Checked aria-live regions for dynamic content announcements
- Analyzed Tailwind color configuration for WCAG AA contrast ratios (4.5:1 minimum)
- Searched for skip navigation links and landmark regions (<nav>, <main>, <aside>, <header>)
- Identified custom modal dialogs vs. Radix UI dialogs (which provide focus traps by default)
- Ran Google Lighthouse v13 accessibility audits on all 11 major pages (Login, Dashboard, My Week, Issues, Projects, Programs, Documents, Team Allocation, Team Directory, Team Status, Settings) using headless Chrome with authenticated sessions via CDP (Chrome DevTools Protocol)
- Categorized Lighthouse violations by severity: Critical (page-level ARIA violations), Serious (color-contrast, listitem), Moderate (minor ARIA issues), and counted total violations per category
- Assessed keyboard navigation completeness: tested Tab/Shift+Tab traversal, Enter/Escape for modals, j/k for list navigation, arrow keys for trees, Cmd+K command palette
- Reviewed screen reader compatibility via code analysis: sr-only text (14 instances), aria-live regions (14), aria-label (114), role attributes (90+), semantic landmarks

Audit Deliverable

Metric	Baseline
Total ARIA attribute instances	~200+ across web/src/
aria-label instances	114 across 43 files
aria-live regions	14 across 10 files
role= attributes	~90+ across 30+ files
Skip navigation link	Present (App.tsx, sr-only with focus:not-sr-only)
Landmark regions	nav (5), main (3), aside (2), header (multiple) — no footer

Keyboard navigation handlers	23 onKeyDown handlers across 17 files
Focus management instances	50+ (useRef + .focus(), useFocusOnNavigate hook)
Screen reader text (sr-only)	14 instances across 8 files
Custom modals lacking focus traps	3 (ConversionDialog, MergeProgramDialog, BacklogPickerModal)
Form inputs missing labels	4-6 in AdminWorkspaceDetail.tsx
Accent color contrast vs background	3.6:1 (#005ea2 on #0d0d0d) — FAILS AA for text
E2E accessibility test pages covered	4 of 15+ pages (login, app shell, docs, programs)
Total Critical violations (Lighthouse)	2 (aria-required-children on Login + Documents pages)
Total Serious violations (Lighthouse)	15 (4 color-contrast on Dashboard + 9 on My Week + 2 listitem)
Keyboard navigation completeness	Partial — j/k navigation, Cmd+K palette, arrow keys for trees/grids; 3 custom modals lack focus traps (Tab escapes to background)
Color contrast failures	13 elements across 2 pages (Dashboard: 4, My Week: 9) — text on muted/accent colors
Missing ARIA labels or roles	<nav> in AdminDashboard.tsx and WorkspaceSettings.tsx lack aria-label; 4-6 <select>/<input> in AdminWorkspaceDetail.tsx lack labels

Lighthouse Accessibility Scores (per-page audit):

Page	Score	Failures
Login	91	aria-required-children (1, critical), listitem (1, serious)
Dashboard	96	color-contrast (4 elements, serious)
My Week	96	color-contrast (9 elements, serious)
Issues	100	None
Projects	100	None
Programs	100	None
Documents	91	aria-required-children (1, critical), listitem (1, serious)
Team Allocation	100	None
Team Directory	100	None
Team Status	100	None
Settings	100	None

Average Lighthouse accessibility score: 97.4/100. 7 of 11 pages score a perfect 100. The lowest-scoring pages are Login (91) and Documents (91), both with critical aria-required-children violations. Dashboard and My Week score 96 due to color-contrast issues.

Accessibility strengths:

- Comprehensive ARIA coverage with proper role patterns (tree, grid, menu, dialog, combobox)
- WCAG-compliant color system with documented contrast ratios (foreground #f5f5f5 on #0d0d0d = 18.4:1)
- Muted text explicitly adjusted from 4.09:1 to 5.1:1 to pass AA
- Superhuman-style keyboard navigation (j/k, arrow keys, Cmd+K command palette)
- Focus management on route navigation via useFocusOnNavigate hook
- Radix UI primitives provide accessible dialogs, popovers, tooltips out of the box
- USWDS icon library with proper aria-hidden/aria-label patterns
- Existing accessibility test suite with axe-core integration (68 test cases total)

Findings and Severity

[HIGH] 3 custom modal dialogs lack focus traps: ConversionDialog.tsx, MergeProgramDialog.tsx, and BacklogPickerModal.tsx implement custom dialog UI with role="dialog" and aria-modal="true" but no focus trap. When these dialogs open, Tab key can escape to background content, violating WCAG 2.4.3 (Focus Order). The Radix Dialog used elsewhere provides focus traps automatically — these should be migrated.

[HIGH] Accessibility tests cover only 4 of 15+ pages: axe-core scans only test login, app shell, docs mode, and programs. Issues, Projects, Settings, Team, Editor, Kanban board, and all modal dialogs have zero automated accessibility testing. A single uncaught ARIA violation on a high-traffic page would break 508 compliance.

[MEDIUM] 4-6 form inputs missing labels in AdminWorkspaceDetail.tsx: Several <select> elements for member roles and <input> fields lack associated <label> elements or aria-label attributes. Screen readers cannot convey the purpose of these controls.

[MEDIUM] Accent color (#005ea2) fails WCAG AA for text: The accent color has a 3.6:1 contrast ratio against the dark background (#0d0d0d), below the 4.5:1 minimum for normal text. If this color is used for text content (not just focus outlines), it fails AA compliance.

[MEDIUM] Multiple <nav> elements without unique aria-labels: AdminDashboard.tsx and WorkspaceSettings.tsx have <nav> elements without aria-label. When multiple navigation landmarks exist, screen readers cannot distinguish between them without unique labels.

[LOW] No <footer> landmark anywhere in the application: While not strictly required, a footer landmark helps screen reader users navigate to copyright/legal/contact information.

Improvement Target

Fix all Critical/Serious violations on the 3 most important pages. Priorities: (1) Add focus traps to the 3 custom modal dialogs (migrate to Radix Dialog or implement trap), (2) Extend axe-core E2E tests to cover Issues, Projects, and Team pages, (3) Add missing form labels in AdminWorkspaceDetail.tsx. Target: 10+ point Lighthouse accessibility score improvement on the lowest-scoring page.

Summary: Audit Baseline Overview

This audit establishes baseline measurements across all 7 categories. The Ship codebase demonstrates strong engineering practices in many areas (strict TypeScript, comprehensive testing, thoughtful accessibility), while having clear improvement opportunities in bundle optimization, database query efficiency, and server-side error handling.

Category	Key Baseline Metric	Top Finding	Severity
1. Type Safety	503 total violations (98 any, 158 as any, 222 as Type)	Test files dominate violations	MEDIUM
2. Bundle Size	2,073 KB main bundle (589 KB gzipped)	Monolithic bundle, no route-level code splitting	CRITICAL
3. API Response Time	Issues: P50=34-45ms, P95=83-109ms at 10-50 conn	Issues + docs endpoints P95 > 80ms under load	HIGH
4. DB Query Efficiency	8 correlated subqueries/row in weeks	Missing JSONB expression indexes	CRITICAL
5. Test Coverage	1,484 tests (882 E2E + 451 API + 151 web unit)	@vitest/coverage-v8 not installed; web coverage not configured	HIGH
6. Error Handling	No process-level error handlers	Server crashes on unhandled rejection	CRITICAL
7. Accessibility	Lighthouse avg 97.4/100, 7/11 pages at 100	Login & Docs at 91 (critical), color-contrast on 2 pages	HIGH

Priority ranking for improvement phase (by impact and severity):

1. P0 (Critical): Add process-level error handlers and Express global error middleware
2. P0 (Critical): Refactor correlated subqueries in weeks endpoint + add JSONB expression indexes
3. P1 (High): Implement route-level code splitting to reduce initial bundle by 20%+
4. P1 (High): Add focus traps to 3 custom modals + extend accessibility test coverage
5. P1 (High): Configure web code coverage + add offline resilience tests
6. P2 (Medium): Replace `as any` in test files with typed mock helpers (25% type safety reduction)
7. P2 (Medium): Add Zod validation to unvalidated route handlers